Software Test Plan (STP) - Smart Parking Management System

Project: Smart Parking Management System (SPMS)
Version: 1.1
Authors: Team 06 – Raghav G Uddihal, Prithvish Adhikari, Rahul Prasanna, Rufus Thomas
Date: 08-11-2026
Status: Final – updated for Sprint 1-2 changes

1. Introduction

Purpose: This document defines the test plan for the Smart Parking Management System (SPMS) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing.

Scope: Testing covers SPMS features such as registration/login, lot and slot management, real-time availability, slot reservation, automatic slot allocation, vehicle entry/exit, fee calculation and payments, reports and notifications. Physical gate hardware (barriers, ANPR cameras, sensors) and payment gateway internals are excluded.

References: SPMS SRS v1.0, SPMS SAD (Design Specification) v1.0, OWASP ASVS 4.0, WCAG 2.1.

Definitions: SPMS (Smart Parking Management System), JWT (JSON Web Token), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing).

2. Test Items

- Authentication & user module
- Lot, slot & tariff management module
- Real-time availability module
- Reservation module
- Slot allocation engine
- Vehicle entry/exit (gate) module
- Billing & payment module
- Reports, history & notification module
- Admin dashboard / attendant console UI

3. Features to be Tested

Features mapped to SRS requirement IDs:

- SPMS-F-001: Driver registration
- SPMS-F-002: Login & JWT issue
- SPMS-F-003: Role-based access control
- SPMS-F-004: Account lockout after 5 failures

- SPMS-F-010: Manage lots / levels / slots
- SPMS-F-011: Configure tariffs
- SPMS-F-012: Slot maintenance status
- SPMS-F-020: Display free-slot counts
- SPMS-F-021: Live status push ≤ 5 s
- SPMS-F-030: Reserve a slot
- SPMS-F-031: Prevent double booking
- SPMS-F-032: Modify / cancel with refund
- SPMS-F-033: Auto-release no-shows
- SPMS-F-040: Auto-allocate suitable slot
- SPMS-F-041: Attendant allocation override
- SPMS-F-050: Record vehicle entry
- SPMS-F-051: Validate reservation QR
- SPMS-F-052: Reject duplicate entry
- SPMS-F-053: Record vehicle exit
- SPMS-F-060: Fee calculation
- SPMS-F-061: Online & cash payment
- SPMS-F-062: Digital receipt
- SPMS-F-063: Block unpaid exit / waiver
- SPMS-F-070: Occupancy & revenue dashboard
- SPMS-F-071: CSV export
- SPMS-F-072: Email notifications
- SPMS-F-073: Driver history & receipts
- SPMS-NF-001: API response time
- SPMS-NF-002: Gate transaction time
- SPMS-NF-003: Availability & backup
- SPMS-NF-004: Usability & accessibility
- SPMS-NF-005: Scalability
- SPMS-NF-006: Maintainability

4. Features Not to be Tested

- Payment gateway internal processing and settlement (vendor responsibility; tested only in sandbox mode)
- Physical hardware – boom barriers, ANPR cameras, slot sensors (out of scope, simulated via API)
- Email delivery beyond the SMTP server (spam filtering / inbox placement)

5. Test Approach / Strategy

Levels:
- Unit tests (module-level, pytest)
- Integration tests (API ↔ database through FastAPI TestClient, API ↔ mock payment gateway)
- System tests (end-to-end user journeys SYS-01…06 through the HTTP API, tests/test_system_e2e.py; UI walk-through by hand)
- Acceptance tests (UAT)

Types:
- Functional testing (core features, boundary values: grace period, daily cap, reservation window)
- Concurrency testing (parallel bookings for the last slot – no double booking)
- Regression testing (CI on every pull request)
- Performance testing (response time, load with 200 concurrent users)
- Usability testing (UI clarity, accessibility)

Entry Criteria: Stable build delivered, test data available, test environment ready.
Exit Criteria: 100% of planned test cases executed, 0 critical defects open, all acceptance criteria satisfied.

5.1 Security Validation

- Validate password handling (bcrypt hashing, not logged) and account lockout
- TLS 1.2+ verification (deployment; local demo runs on http – deviation D-08)
- JWT expiry/tampering and role-based access (RBAC) checks
- Payment webhook signature verification; no card data stored
- Fuzzing for input fields (plate number, login, reservation forms) – SQL injection / XSS (automated security suite; OWASP ZAP baseline optional)
- Penetration testing of authentication flows and rate limiting

6. Test Environment

Hardware: Developer laptops (Windows 11) and GitHub Actions runners (ubuntu-latest).
Software: SPMS v1.0 (FastAPI + plain HTML/JS pages), SQLite database, mock payment gateway (test mode), email outbox (optional SMTP sandbox), Python 3.12.
Tools: pytest + FastAPI TestClient (unit, integration, system), pytest-cov (line and branch coverage), Locust (performance, perf/locustfile.py), tools/test_reports.py (report generation), GitHub Issues and Project board (defect tracking), GitHub Actions (CI on every pull request).
Test Data: tests/conftest.py builds a fresh database for every test; python -m backend.seed (demo lot, accounts); backend/volume_seed.py (10 lots / 5,000 slots / 1,000 bookings).

7. Test Schedule

Milestones (planned → actual):
- Test case design: 20-Oct-2026 → 20-Oct-2026 (IDs), detailed specs 08-Nov-2026
- Environment setup: 24-Oct-2026 → 10-Oct-2026 (CI from the first PR)
- Test execution: 02-Nov to 13-Nov-2026 → continuous in CI on every PR from 29-Oct-2026; formal cycle 07-Nov to 08-Nov-2026
- UAT: 16-Nov-2026 to 18-Nov-2026 (unchanged)

8. Test Deliverables

- Test Plan (this document, v1.1)
- Test Cases (section 13.1)
- Test Scripts (tests/, perf/locustfile.py)
- Test Data (tests/conftest.py, backend/seed.py, backend/volume_seed.py)
- Test Execution Log, Unit Test Report, System Validation Report, Code Coverage and Branch

Coverage Reports, RTM (docs/testing/, generated by tools/test_reports.py)
- Defect Report and Security Test Report (docs/testing/)
- Test Summary Report (docs/testing/Test_Summary_Report.md)

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Rahul Prasanna	Prepare plan, coordinate execution
Test Engineer	Rufus Thomas	Design & execute test cases, log defects
Developer	Raghav G Uddihal	Support defect fixes and triage
Product Owner	Prithvish Adhikari	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early smoke builds from dev team
Test environment downtime	Run the full stack locally with Docker Compose as backup
Payment gateway sandbox unavailable	Use a mock gateway with the same API and signed webhooks
Concurrency defects hard to reproduce	Automated parallel-request tests and database constraints

11. Assumptions & Dependencies
- Payment gateway sandbox and SMTP sandbox will be stable and available
- Test data (users, vehicles, lots, slots, tariffs) will be seeded before execution
- Requirements are frozen at SRS v1.0 for this test cycle

12. Suspension & Resumption Criteria
Suspend testing if:
- Environment unavailable for >4 hours
- Build is too unstable (blocks >30% test cases)

Resume testing if:

- Blocking defects are resolved
- Environment stabilized

13. Test Case Management & Traceability

RTM ensures mapping of SRS requirements to test cases.
Example:
- SPMS-F-001 (Driver registration) → TC-AUTH-01
- SPMS-F-002 (Login & JWT issue) → TC-AUTH-02
- SPMS-F-003 (Role-based access control) → TC-AUTH-03
- SPMS-F-004 (Account lockout after 5 failures) → TC-AUTH-04
- SPMS-F-010 (Manage lots / levels / slots) → TC-LOT-01
- SPMS-F-011 (Configure tariffs) → TC-LOT-02
- SPMS-F-012 (Slot maintenance status) → TC-LOT-03
- SPMS-F-020 (Display free-slot counts) → TC-AVL-01
- SPMS-F-021 (Live status push ≤ 5 s) → TC-AVL-02
- SPMS-F-030 (Reserve a slot) → TC-RES-01
- SPMS-F-031 (Prevent double booking) → TC-RES-02
- SPMS-F-032 (Modify / cancel with refund) → TC-RES-03
- SPMS-F-033 (Auto-release no-shows) → TC-RES-04
- SPMS-F-040 (Auto-allocate suitable slot) → TC-ALC-01
- SPMS-F-041 (Attendant allocation override) → TC-ALC-02
- SPMS-F-050 (Record vehicle entry) → TC-GATE-01
- SPMS-F-051 (Validate reservation QR) → TC-GATE-02
- SPMS-F-052 (Reject duplicate entry) → TC-GATE-03
- SPMS-F-053 (Record vehicle exit) → TC-GATE-04
- SPMS-F-060 (Fee calculation) → TC-PAY-01
- SPMS-F-061 (Online & cash payment) → TC-PAY-02
- SPMS-F-062 (Digital receipt) → TC-PAY-03
- SPMS-F-063 (Block unpaid exit / waiver) → TC-PAY-04
- SPMS-F-070 (Occupancy & revenue dashboard) → TC-RPT-01
- SPMS-F-071 (CSV export) → TC-RPT-02
- SPMS-F-072 (Email notifications) → TC-NOT-01
- SPMS-F-073 (Driver history & receipts) → TC-HIS-01
- SPMS-NF-001 (API response time) → TC-PERF-01
- SPMS-NF-002 (Gate transaction time) → TC-PERF-02
- SPMS-NF-003 (Availability & backup) → TC-REL-01
- SPMS-NF-004 (Usability & accessibility) → TC-UX-01
- SPMS-NF-005 (Scalability) → TC-PERF-03
- SPMS-NF-006 (Maintainability) → TC-MNT-01
- SPMS-SR-001 (TLS 1.2+ everywhere) → TC-SEC-01
- SPMS-SR-002 (bcrypt password hashing) → TC-SEC-02
- SPMS-SR-003 (JWT expiry & revocation) → TC-SEC-03
- SPMS-SR-004 (Input validation / no SQLi-XSS) → TC-SEC-04
- SPMS-SR-005 (No card data; signed webhooks) → TC-SEC-05

- SPMS-SR-006 (Rate limiting) → TC-SEC-06
- SPMS-SR-007 (Append-only audit log) → TC-SEC-07

13.1 Test Case Specifications (v1.1)

Each test case below is automated unless marked manual. Every automated test carries its test-case ID in a comment (# TC-...), so docs/testing/RTM.md is generated from the code and shows the result of the latest run.

Test case	Req.	Level	Steps / data	Expected result	Automated in (tests/)
TC-AUTH-01	SPMS-F-001	Unit / API	POST /auth/register with name, email, 10-digit phone, password (>= 6 chars); repeat with the same email; try invalid email/phone.	201 and role DRIVER; duplicate email -> 409; invalid input -> 422.	test_auth.py, test_security_suite.py
TC-AUTH-02	SPMS-F-002	Unit / API	POST /auth/login with correct and with wrong credentials.	200 with access token (30 min), refresh token and role; wrong password or unknown email -> 401 with the same message.	test_auth.py, test_security_suite.py
TC-AUTH-03	SPMS-F-003	API / Security	Call 12 protected endpoints as anonymous, driver, attendant and admin.	401 without login, 403 for a role that is not allowed, success for allowed roles.	test_audit_security.py, test_auth.py, test_gate.py, test_login_redirect.py, test_lots.py, test_override.py, test_payments.py, test_reports.py, test_reservation_changes.py, test_reservations.py, test_security_suite.py
TC-AUTH-04	SPMS-F-004	Unit / API	Log in 5 times with a wrong password, then with the right one; advance the clock 15 min.	6th attempt -> 423 Locked; after 15 minutes login works again.	test_auth.py
TC-LOT-01	SPMS-F-010	API	Admin creates a lot, a level and slots in bulk; edits the lot; a driver tries the same.	201/200 for admin; duplicate lot name -> 409; driver -> 403; slots appear in availability.	test_audit_security.py, test_lots.py, test_system_e2e.py
TC-LOT-02	SPMS-F-011	API	Admin sets a tariff (hourly rate, grace, daily cap) per vehicle type.	Tariff stored and used for the next fee; change written to the audit log.	test_audit_security.py, test_lots.py
TC-LOT-03	SPMS-F-012	API / System	Admin puts slot A-1 into maintenance, then a car enters.	Slot shows MAINTENANCE, is not counted as free and is never allocated.	test_audit_security.py, test_lots.py, test_system_e2e.py
TC-AVL-01	SPMS-F-020	Unit / API	Seed slots of each type, occupy and reserve some, GET /lots/{id}/availability.	Free counts per type and level equal the database state; maintenance slots excluded.	test_availability.py, test_availability_api.py, test_reservations.py, test_system_e2e.py
TC-AVL-02	SPMS-F-021	API (WebSocket)	Open ws /ws/lots/{id}/availability, then log an entry.	A new snapshot arrives within 5 s (sent every 2 s); unknown lot closes with code 1008.	test_availability_api.py
TC-RES-01	SPMS-F-030	API / System	Driver adds a vehicle and reserves 10:00-11:00; try a start in the past, < 30 min, > 7 days ahead.	201 with code, slot, advance (full tariff) and QR token; invalid windows -> 422.	test_payments.py, test_reservations.py, test_system_e2e.py
TC-RES-02	SPMS-F-031	Concurrency / System	50 parallel requests for the last free slot; same vehicle books two overlapping windows.	Exactly one succeeds, the others get 409; no slot is ever double-booked.	test_reservations.py, test_system_e2e.py

TC-RES-03	SPMS-F-032	API / System	Cancel >= 1 h before start, cancel < 1 h before, modify the time, cancel after start.	100 % refund, 50 % refund, advance recalculated; after start -> 409; slot freed.	test_reservation_changes.py, test_system_e2e.py
TC-RES-04	SPMS-F-033	API	Confirmed booking, no check-in; advance the clock past start + 15 min.	Status NO_SHOW and the slot is free for others.	test_reservation_changes.py, test_reservations.py
TC-ALC-01	SPMS-F-040	Unit / System	Walk-ins and bookings for car, EV and accessible vehicles.	Nearest free compatible slot; accessible need gets an H slot; slots reserved within the next hour avoided.	test_gate.py, test_reservations.py, test_system_e2e.py
TC-ALC-02	SPMS-F-041	API	Attendant moves ticket T-000001 to another slot with a reason; try an occupied/incompatible slot.	Car moved, audit row with reason; occupied or incompatible slot -> 409; drivers -> 403.	test_override.py
TC-GATE-01	SPMS-F-050	API / System	Attendant logs entry for plate KA01AB1234; invalid plate; full lot.	201 with ticket and slot; invalid plate -> 422; full lot -> 409.	test_gate.py, test_system_e2e.py
TC-GATE-02	SPMS-F-051	API / System	Entry with the booking's QR inside +-15 min; with an expired, forged or other-plate QR.	Car goes to its reserved slot; invalid, expired or other-plate QR -> 400.	test_gate.py, test_history_qr_window.py, test_reports.py, test_reservations.py, test_system_e2e.py
TC-GATE-03	SPMS-F-052	API / System	Log entry twice for the same plate.	Second entry -> 409.	test_gate.py, test_system_e2e.py
TC-GATE-04	SPMS-F-053	API / System	Log exit after 10, 15 and 70 minutes; exit an unknown plate.	Exit time, minutes and fee recorded, slot freed; unknown plate -> 404.	test_gate.py, test_payments.py, test_system_e2e.py
TC-PAY-01	SPMS-F-060	Unit	Fee for 15, 16, 70 min and 30 h, with a prepaid advance.	<= grace free; each started hour charged; daily cap applied; advance deducted.	test_billing.py, test_gate.py, test_payments.py, test_system_e2e.py
TC-PAY-02	SPMS-F-061	API / System	Pay the due amount in cash; pay online through the mock gateway (UPI/card); forge a confirmation.	Payment PAID and exit allowed; forged signature -> 400.	test_payments.py, test_system_e2e.py
TC-PAY-03	SPMS-F-062	API / System	Exit after payment, open the receipt JSON and PDF; another driver opens it.	Receipt with number, times, fee, advance, method; valid PDF; other driver -> 403/404.	test_payments.py, test_system_e2e.py
TC-PAY-04	SPMS-F-063	API / System	Try to exit with fee unpaid; attendant then admin waives with a reason.	Exit -> 402 with amount; attendant waiver -> 403; admin waiver allowed and audited.	test_payments.py, test_system_e2e.py
TC-RPT-01	SPMS-F-070	API / System	Seed a day of entries, payments and refunds; GET /reports/summary for that day.	Occupancy %, revenue (payments + advances - refunds), average stay and peak hours equal a plain SQL check.	test_reports.py, test_system_e2e.py
TC-RPT-02	SPMS-F-071	API / System	GET /reports/export.csv for the same range.	CSV with one row per lot and a TOTAL row equal to the dashboard totals.	test_reports.py, test_system_e2e.py
TC-NOT-01	SPMS-F-072	API / System	Confirm, cancel and approach a booking start; set SPMS_SMTP_HOST.	Confirmation, cancellation and 15-min reminder emails in the outbox; sent through SMTP when configured.	test_reservation_changes.py, test_system_e2e.py
TC-HIS-01	SPMS-F-073	API / System	Driver opens history; a second driver opens theirs; driver 10 min late opens	Own vehicles, bookings, sessions and receipts only; QR still shown	test_history_qr_window.py, test_reports.py,

			the QR.	until start + 15 min (DEF-01).	test_system_e2e.py
TC-PERF-01	SPMS-NF-001	Performance	Time 200 mixed API calls in-process; manual Locust run with 200 users (perf/locustfile.py).	p95 response time < 2 s.	test_performance.py
TC-PERF-02	SPMS-NF-002	Performance	Time entry, exit and payment at the gate.	Each gate transaction < 3 s.	test_performance.py
TC-PERF-03	SPMS-NF-005	Volume	Seed 10 lots / 5,000 slots / 1,000 bookings (backend/volume_seed.py), run availability and booking.	Works without schema change; responses stay within NF-001.	test_scalability.py
TC-REL-01	SPMS-NF-003	Reliability	Make a backup, change data, restore; make 8 backups.	Data restored exactly; only the newest 7 backups kept; admin-only API.	test_backup.py, test_health.py
TC-UX-01	SPMS-NF-004	Usability / Accessibility	Check every page: labels, headings, language, contrast, focus ring, announcements, reserve in <= 3 screens.	WCAG 2.2 AA checks pass; a driver reserves within 3 screens.	test_payments.py, test_ui.py, test_ui_accessibility.py, test_usability.py
TC-MNT-01	SPMS-NF-006	Static / CI	Run the suite with coverage in CI on every pull request.	Line coverage >= 70 % or the PR cannot merge.	CI coverage gate
TC-SEC-01	SPMS-SR-001	Manual (deployment)	Inspect the deployed URL with a TLS checker.	TLS 1.2+ only. Local demo runs on http (deviation D-08).	manual
TC-SEC-02	SPMS-SR-002	Security	Register, inspect the database and API responses; compare login errors.	bcrypt hash, cost 12; hash never returned; no account enumeration.	test_auth.py, test_auth_tokens.py, test_security_suite.py
TC-SEC-03	SPMS-SR-003	Security	Use expired, unsigned (alg none), wrongly signed, edited and logged-out tokens.	All refused with 401; role always read from the database.	test_auth.py, test_auth_tokens.py, test_security_suite.py
TC-SEC-04	SPMS-SR-004	Security	Send SQL-injection strings, <script> and path tricks in forms and URLs.	Rejected (422) or harmless; pages escape output; security headers present.	test_audit_security.py, test_security_suite.py
TC-SEC-05	SPMS-SR-005	Security	Inspect tables and the gateway page; forge a payment confirmation.	No card data stored or asked for; forged HMAC -> 400.	test_payments.py, test_security_suite.py, test_system_e2e.py
TC-SEC-06	SPMS-SR-006	Security	11 logins or registrations within a minute from one address.	11th request -> 429 with Retry-After.	test_auth_tokens.py, test_security_suite.py
TC-SEC-07	SPMS-SR-007	Security	Change a tariff, toggle maintenance, override, waive; try to edit/delete audit rows.	Each action logged with who/when/before/after/reason; rows cannot be changed.	test_audit_security.py, test_system_e2e.py

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed/failed
- Defect density
- Defect aging
- Requirement coverage

Reports:
- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	Rahul Prasanna	Rahul/11-10-2026
Test Engineer	Rufus Thomas	Rufus/11-10-2026
Dev Lead	Raghav G Uddihal	Raghav/11-10-2026
Product Owner	Prithvish Adhikari	Prithvish/11-10-2026